

Analysis of Model Predictions and Strategy

Unrealistic Extreme Predictions: Causes

The model is outputting **extreme 20-minute return forecasts** that are clearly unrealistic (e.g. predicting +30% or even +44% moves in 20 minutes for stocks like AGNC) ¹. This likely indicates something went wrong in the prediction mechanism. One cause is the **compounding effect** of the model's minute-by-minute predictions. The strategy rolls the model forward 20 steps, and if the model even slightly overestimates each minute's return, those small errors compound into a huge predicted move. For instance, a ~1.5% predicted rise per minute sustained over 20 minutes compounds to ~35% – a scenario virtually never seen in normal trading. These extreme outputs suggest the model may be **overfitted or miscalibrated**, latching onto noise and exaggerating trends. It could also indicate an **issue with normalization or constraints** – the model isn't bounded in its predictions, so nothing prevents implausibly large returns. In short, the model's predictions appear **random and overly confident**, which is why you see impossible swings being forecast. Ensuring the model is better regularized (or using a capped activation like tanh for returns) and representing realistic volatility in training could help mitigate this.

Another factor is that **uncertainty estimates did not temper the predictions**. The Monte Carlo dropout approach was supposed to give a distribution of outcomes, but in practice it gave nearly identical outcomes every time. This is evident from the metrics: every prediction had $p_netpos \approx 1.000$ and entropy ≈ 0 ², meaning the model was effectively 100% sure of net profit with no uncertainty. Such confidence is a red flag – it implies either the dropout isn't functioning (perhaps the model has no dropout layers or they weren't active) or the model outputs the same path for all samples. With no variation in the simulated paths, the mean return (μ_{20}) stays extreme and the **predicted standard deviation** (σ_{20}) **stays tiny**, giving a huge signal-to-noise. In summary, the model is likely **too certain and too aggressive** in its forecasts due to compounding errors and lack of proper uncertainty calibration.

SNR Metric and Its Usage

You noticed the **SNR (Signal-to-Noise Ratio)** appearing in the outputs even though it didn't seem to influence trades. In the current implementation, the classic SNR threshold (μ/σ) isn't explicitly used to trigger trades; instead the code switched to using the **probability of net profit (p_netpos)** and entropy as the confidence measures. The variable snr in the trade logs is actually just an **alias for p_netpos** for legacy reasons ³. So every trade shows $SNR: 1.00$ simply because the model's p_netpos was ~1 (100% probability of profit) for those entries. The strategy was originally documented as SNR-based, but it evolved – now trades require conditions like $\mu_{20} - k \cdot risk \cdot \sigma \geq cost$ (a risk-adjusted edge) and $p_netpos \geq 0.65$ with low entropy, instead of a raw SNR cutoff. The **SNR categories in the output are essentially vestigial** – the code still prints them for analysis, but they didn't affect decisions. This is why the trade categorization shows “No trades” for $SNR > 1/2/3$: all trades were logged with an snr value of 1.0 (meaning $p_netpos \sim 1$), so none exceeded those thresholds in the log. In short, **SNR remains in the output for debugging/reference**, but the strategy logic now relies on the probability-based confidence measure.

Assessment of the Prediction+Entropy Strategy

The strategy of using **prediction magnitude plus entropy (confidence)** is sound in principle, but in practice it didn't behave as expected here. The implementation indeed checks the 20-min predicted return (μ_{20}), its uncertainty (σ_{20}), the probability of a positive outcome (p_{netpos}), and the entropy of that probability. For entry, the code requires $p_{\text{netpos}} \geq 0.65$ and entropy ≤ 0.8 bits, plus a risk-adjusted return above cost and agreement in short-term direction ⁴ ⁵. For exits, it uses thresholds like $p_{\text{netpos}} < 0.55$ or entropy > 1.0 to bail out if confidence decays, and it sets stop/take-profit levels based on σ_{20} and μ_{20} ⁶ ⁷. So the **mechanics are implemented correctly as per the design**. The problem is that the model's outputs never really exercised these controls. Because **p_{netpos} was effectively 1.0 and entropy 0 for virtually every prediction**, every signal sailed through the entry criteria with maximum confidence. The entropy-threshold didn't help distinguish good vs. bad signals because the model always reported zero entropy (total certainty). Similarly, the exit conditions based on confidence never triggered early – the model never indicated doubt (p_{netpos} never fell below 0.55 while a trade was open), and the strategy also didn't stop out losing positions due to the built-in “spread guard” requiring break-even before exit ⁸ ⁹. This is why almost all positions simply closed at the 20-minute horizon, regardless of minor losses. In summary, the **prediction + entropy strategy was coded properly, but it failed to improve results because the model's confidence metrics were skewed** (always maximal), effectively reducing the strategy to “always trust the prediction and hold for 20 minutes.” The intent was to only trade when the model was confident and to exit if uncertainty rose, but since the model was *always* “confident” (perhaps due to a bug or lack of dropout effect), the strategy's nuanced logic never had a chance to come into play.

Suggestions for Improvement

Given the above issues, here are a few suggestions to improve the model and strategy:

- 1. Calibrate and Constrain the Model Predictions:** The model should reflect realistic market moves. Introduce regularization or output constraints to prevent absurd predictions. For example, use a **bounded activation** (like $\tanh$) at the output to limit the minute-level return predictions to a reasonable range, or explicitly penalize large errors so the model doesn't learn to output spikes. Ensuring the training data is scaled and normalized properly is crucial – if there's a scale mismatch, fix the normalization so that a 0.44 output truly means +44% (which the model should rarely if ever predict). By curbing extreme outputs, the model's forecasted μ_{20} will be more plausible, avoiding the random wild swings you're seeing.
- 2. Improve Uncertainty Estimation (Dropout/Ensembles):** The fact that p_{netpos} stayed at 1.0 and entropy at 0 indicates the uncertainty mechanism isn't working. Make sure the model has dropout layers (or use an ensemble of models) so that Monte Carlo simulations produce diverse outcomes. Currently, it appears every MC sample yields the same path, so incorporate **stochastic elements** – e.g. add dropout in the LSTM/FC layers or use a Bayesian approach – to get a spread of predictions. This will give p_{netpos} values that vary between, say, 0.5 and 0.99 instead of always 1.0, and non-zero entropy. With a more truthful measure of confidence, the strategy's filters (like requiring $p_{\text{netpos}} \geq 65\%$) will actually screen out low-confidence signals. In short, **fix the dropout usage or adopt a better uncertainty model** so the strategy isn't blind to risk.

3. **Adjust Entry/Exit Thresholds and Logic:** Revisit the trade criteria in light of transaction costs and signal quality. A 0.10% edge threshold for entry might be too low given the 0.10% round-trip cost – essentially you’re trading on signals that barely cover fees, leading to flat or negative returns after cost. Consider **raising the entry threshold** (e.g. require μ_{20} at least 0.2–0.3% above cost or $p_{\text{netpos}} \gg 0.65$) so you only act on stronger predictions. Likewise, implement a more proactive exit for bad trades: the current strategy avoids selling at a loss (to not lock in the spread cost), but this can backfire if a prediction was wrong. It may be better to **allow a stop-loss even if it means a small guaranteed loss**, to prevent a small loss from potentially growing. For example, if the model’s signal flips or confidence drops sharply, consider cutting the loss instead of waiting the full 20 minutes. Fine-tuning the **hysteresis between entry and exit** (e.g. enter on $>0.3\%$ prediction, exit if falls below 0.1%) can also prevent whipsaw trades while still protecting profits.
4. **Smooth or Filter the Signals:** If the model’s directional accuracy is good overall but jittery, you can impose some smoothing on the trading decisions. For instance, require that a buy signal persists for several minutes before entry, or use a short moving average of μ_{20} to confirm the trend. This avoids reacting to one-minute prediction noise. Additionally, since you limit to top 3 trades per minute, ensure you’re not always picking the same few stocks due to tie-breaking. If all p_{netpos} are equal, maybe sort by predicted **magnitude (μ_{20})** to choose the truly best opportunities, or allow more parallel trades if many signals are strong. These tweaks make the strategy more robust to momentary random fluctuations in the predictions.
5. **Re-evaluate Model Training for Directional Bias:** It’s concerning that the model almost never predicted negative moves during this window. Verify that the training process isn’t biased towards predicting upward moves (for example, if the training data had an upward drift or if loss function asymmetrically penalized errors). If needed, retrain the model with a focus on capturing downtrends as well, or even use a separate classifier to predict direction (up/down) alongside magnitude. A more balanced model that sometimes predicts downward movement would allow the strategy to go to cash or short when appropriate, instead of always being long. Incorporating the ability to **short sell on strong negative predictions** (if allowed) could also improve performance, turning the model’s directional skill into profits on both sides.

By implementing these improvements, you should see the model stop making random extreme predictions and instead give more realistic forecasts with meaningful confidence levels. In turn, the strategy can then reliably buy only when there’s a true edge and exit or even short when the tide turns, hopefully yielding better overall returns.

1 2 3 4 5 6 7 8 9 strategy.py

file:///file-KvNhjSvRNie2jd9DruXESi